

Tôi là Tech Lead của dự án ebook manager. Dựa trên tài liệu Đặc tả phiên bản 2.0, chúng ta sẽ nâng cấp hệ thống để sẵn sàng thương mại hóa. Hãy đọc kỹ các yêu cầu nâng cấp dưới đây cho TDD-010, TDD-013, TDD-015 và module mới TDD-020. Viết mã Python/PySide6 tương ứng để thực thi.

1. Nâng cấp TDD-010: Main Window & UI Shell (Bổ sung Menu & Thanh công cụ)

Mục tiêu: Mở rộng giao diện 3-Pane Layout (Sidebar - Header - Content) bằng cách thêm hệ thống Menu Bar chuẩn mực của ứng dụng desktop.

Phạm vi cập nhật:

- **Hệ thống Menu (QMenuBar):**
 - **File:** Thêm tài liệu, Thêm thư mục (Quét), Thoát.
 - **Edit:** Chọn tất cả, Chỉnh sửa thông tin (Mở TDD-020).
 - **View:** Chuyển đổi Grid/List, Bật/Tắt Sidebar, Nhóm tài liệu (Group by: Tác giả, Định dạng).
 - **Tools:** Cài đặt (Settings - TDD-015).
- **Thanh điều khiển bổ sung (ToolBar):** Đặt ngay dưới Omnibar, chứa:
 - Dropdown Sắp xếp (Sort by): Tiêu đề, Tác giả, Ngày thêm, Kích thước file. (Sử dụng lệnh ORDER BY gọi xuống DB).
 - Thanh trượt (QSlider): Điều chỉnh kích thước ảnh bìa (Cover size) mượt mà từ Nhỏ (100px) đến Lớn (300px).

Prompt cho Agent 2 (TDD-010 Cập nhật):

"Viết bổ sung cho class MainWindow (PySide6). Khởi tạo QMenuBar với các menu File, Edit, View, Tools. Tạo một QToolBar chứa QComboBox để chọn tiêu chí sắp xếp (Title, Author, Date, Size) và một QSlider (Orientation.Horizontal) để chỉnh kích thước Cover (range: 100-300). Kết nối tín hiệu thay đổi slider tới một hàm phát Event qua EventBus."

2. Nâng cấp TDD-013: Grid & List View (Phân trang, Nhóm & Kích thước)

Mục tiêu: Xử lý hiển thị tối ưu khi thư viện lên tới hàng chục nghìn cuốn sách bằng Model/View. Bổ sung phân trang (Pagination) và hiển thị theo nhóm (Grouping).

Phạm vi cập nhật:

- **Phân trang (Pagination):**

- UI: Dưới cùng của Content Area thêm cụm điều hướng: [|<] [<] Trang 1 / 50 [>] [>|] và ô nhập số để "Nhảy đến trang".
- Logic: Tích hợp với SQLite TDD-001. Thay vì load toàn bộ, truy vấn dùng LIMIT 100 OFFSET ?.
- **Đồng bộ kích thước Cover:** Nhận tín hiệu từ Slider của TDD-010, gọi lệnh `setViewMode(QListView.IconMode)` và `setIconSize(QSize(val, val*1.5))` để thu phóng ảnh bìa theo thời gian thực.
- **Nhóm tài liệu (Grouping):** Trong chế độ List, sử dụng tính năng của Qt (hoặc proxy model) để hiển thị thanh tiêu đề phân nhóm (Ví dụ: 1 dòng header "Năm 2023", bên dưới là các sách).

Prompt cho Agent 2 (TDD-013 Cập nhật):

"Nâng cấp module hiển thị danh sách (Model/View). Thêm một QWidget chứa các nút Phân trang (First, Prev, Next, Last) và ô nhập trang (Jump). Viết logic tính toán tổng số trang dựa trên tổng record trả về từ DB (chia 100 item/trang). Lắng nghe sự kiện `CoverSizeChangedEvent` từ EventBus để gọi `self.list_view.setIconSize()`. Viết mock dữ liệu để test chuyển trang."

3. TDD-020: Metadata Editor (Tính năng mới - Cho phép chỉnh sửa)

Mục tiêu: Mặc dù hệ thống tự động bóc tách metadata, đôi khi dữ liệu thô bị sai. Người dùng cần một form để sửa thủ công.

Phạm vi & Trách nhiệm:

- UI Form (QDialog): Bật lên khi bấm chuột phải vào sách chọn "Chỉnh sửa thông tin".
- Trường thông tin (Fields): Tiêu đề, Tác giả, Thể loại (Tags), Ngày xuất bản, Kích thước (Read-only), File path (Read-only).
- Ghi đè DB: Lưu thay đổi vào bảng documents và tự động cập nhật bảng ảo documents_fts. Không làm thay đổi nội dung file vật lý (Tuân thủ nguyên tắc Single Source of Truth).

Prompt cho Agent 2 (TDD-020 Mới):

"Viết class `MetadataEditorDialog` kế thừa `QDialog` bằng `PySide6`. Tạo Form Layout chứa các trường `QLineEdit` cho Tiêu đề, Tác giả, và một Token-input (hoặc `QLineEdit`) cho Tags. Khi ấn 'Lưu', tạo payload dictionary gửi xuống hàm `update_document()` của `DatabaseManager`. Nhớ bắn sự kiện `LibraryUpdatedEvent` để Grid/List tự động render lại dữ liệu mới."

4. Nâng cấp TDD-015: Advanced Settings Panel (Cài đặt nâng cao)

Mục tiêu: Bảng cấu hình hệ thống chuyên sâu, hỗ trợ tùy biến phần cứng và lưu trữ JSON tại thư mục %APPDATA%.

Phạm vi cập nhật UI Cài đặt (Settings Dialog):

- **Tab 1 - Quản lý File:** Tick chọn các định dạng cho phép quét tự động (PDF, EPUB, MOBI, AZW3, DOCX).
- **Tab 2 - Giao diện (Theme):** ComboBox chọn phong cách (Light, Dark, Glassmorphism, Cozy).
- **Tab 3 - Tăng tốc phần cứng (Performance):**
 - Cho phép người dùng thiết lập số luồng (Worker Threads) cho Background Job Queue (TDD-007). Tự động gợi ý bằng multiprocessing.cpu_count().
 - Giới hạn dung lượng RAM cache cho ảnh bìa.
- **Tab 4 - Đám mây (Cloud):**
 - Nút "Kết nối Google Drive".
 - Trường nhập ID/Đường dẫn thư mục sao lưu mặc định.
 - Xóa Token (Đăng xuất).

Prompt cho Agent 2 (TDD-015 Cập nhật):

"Mở rộng class ConfigManager và tạo SettingsDialog (sử dụng QTabWidget). Tab 1: Checkbox cho các đuôi file. Tab 2: Theme selector. Tab 3: QSpinBox chọn số Worker Threads (mặc định lấy theo os.cpu_count()). Tab 4: Tích hợp OAuth login/logout Google Drive. Ghi mọi thay đổi vào file settings.json tại thư mục %APPDATA% thông qua os.environ.get('APPDATA')." "

TDD-021: Duplicate Finder & Merger (Hệ thống dò quét trùng lặp)

Mục tiêu: Người dùng thường tải một cuốn sách nhiều lần hoặc lưu ở nhiều thư mục khác nhau. Hệ thống cần tự động phát hiện và dọn dẹp để tiết kiệm ổ cứng, giữ cho thư viện sạch sẽ.

Phạm vi & Trách nhiệm:

- **Thuật toán đối chiếu (Matching Logic):**

- Mức độ 1 (Tuyệt đối): So sánh mã băm (MD5/SHA256) của nội dung file.
- Mức độ 2 (Tương đối): So sánh độ tương đồng của chuỗi Title + Author (Sử dụng thư viện thefuzz hoặc difflib).
- **Giao diện (UI):** Bổ sung mục "Dọn dẹp trùng lặp" trong Menu Tools. Hiển thị một bảng (Table) nhóm các file trùng nhau lại.
- **Xử lý (Action):** Cung cấp nút "Gộp (Merge)" (giữ lại file chất lượng cao nhất, gộp metadata) hoặc "Xóa bản sao" (Xóa file vật lý và xóa khỏi FTS5 Database).

Prompt cho Agent 2 (TDD-021):

"Viết module duplicate_finder.py. Xây dựng class DuplicateEngine có hàm find_exact_duplicates() (query SQL tìm các record có cùng file_size và hash) và find_fuzzy_duplicates() (dùng thư viện difflib để nhóm các sách có tên gần giống nhau). Viết một UI QDialog dạng bảng (QTableWidget) hiển thị danh sách này, có các Checkbox để người dùng chọn file muốn xóa. Kết nối với TDD-014 để xóa file vật lý nếu người dùng xác nhận."

TDD-022: Calibre / External Library Migrator (Công cụ chuyển nhà)

Mục tiêu: Rào cản lớn nhất để bán phần mềm là người dùng đang mắc kẹt dữ liệu ở phần mềm cũ (đặc biệt là Calibre). Chúng ta cần một tính năng "Nhập dữ liệu 1-click".

Phạm vi & Trách nhiệm:

- Cho phép người dùng tro đường dẫn đến thư mục chứa thư viện Calibre hiện tại của họ.
- Đọc trực tiếp file metadata.db (bản chất cũng là SQLite) của Calibre.
- Ánh xạ (Mapping) dữ liệu: Chuyển đổi tags, rating, authors từ cấu trúc của Calibre sang cấu trúc của Document Model (TDD-002) trong SmartDoc.
- Lập chỉ mục toàn bộ số sách đó vào documents_fts mà không cần di chuyển file vật lý đi nơi khác (Tuân thủ nguyên tắc cốt lõi của dự án).

Prompt cho Agent 2 (TDD-022):

"Viết module calibre_migrator.py. Xây dựng class CalibreImporter. Viết hàm nhận đầu vào là file metadata.db của Calibre. Mở kết nối sqlite3 thứ hai (read-only) tới file này, query các bảng books, authors, tags. Viết hàm ánh xạ dữ liệu đọc được sang schema của SmartDoc Library. Sau đó đẩy vào Job Queue (TDD-007) để hệ thống tự động xử lý ngầm, có phát tín hiệu cập nhật Progress Bar."

TDD-023: Bulk Batch Editor (Trình chỉnh sửa hàng loạt)

Mục tiêu: Mở rộng TDD-020 (Metadata Editor) để hỗ trợ chỉnh sửa hàng trăm tài liệu cùng một lúc thay vì phải click vào từng cuốn.

Phạm vi & Trách nhiệm:

- Tại Grid/List View, người dùng có thể bôi đen/chọn (Select) nhiều cuốn sách cùng lúc.
- Chuột phải -> Chọn "Chỉnh sửa hàng loạt".
- Hiển thị UI Form đặc biệt: Các trường thông tin (VD: Tác giả, Thể loại) sẽ có dạng Checkbox bên cạnh. Người dùng tick vào trường nào thì trường đó mới ghi đè cho toàn bộ sách được chọn. Dữ liệu các trường không tick sẽ được giữ nguyên gốc.

Prompt cho Agent 2 (TDD-023):

"Bổ sung tính năng Bulk Edit vào metadata_editor.py. Tạo BatchEditorDialog. Nhận vào danh sách List[doc_id]. Form UI có các trường: Author, Tags, Publisher. Mỗi trường đi kèm một QCheckBox (Ví dụ: [x] Đổi tác giả thành: [____]). Khi bấm Lưu, duyệt qua danh sách doc_id và gọi lệnh UPDATE SQL chỉ trên những cột được check. Đảm bảo chạy trong Transaction (BEGIN/COMMIT) của SQLite để tối ưu tốc độ."

TDD-024: Auto-Updater & Telemetry Engine (Cập nhật tự động)

Mục tiêu: Đảm bảo vòng đời của phần mềm thương mại. Người dùng luôn nhận được bản vá lỗi mới nhất một cách tĩnh lặng mà không cần lên web tải lại file cài đặt.

Phạm vi & Trách nhiệm:

- Kiểm tra phiên bản: Khởi động app -> Chạy một luồng ngầm gọi API (Ví dụ: Github Releases API hoặc server nội bộ của bạn) để check version JSON.
- So sánh version hiện tại (Ví dụ: v2.1.0) với version trên mạng (v2.1.1).
- Nếu có bản mới, hiện thông báo nhỏ góc màn hình (Toast Notification) báo có bản cập nhật.
- Cung cấp tùy chọn "Tải về và Cài đặt lại".

Prompt cho Agent 2 (TDD-024):

"Viết module auto_updater.py. Class UpdateManager chạy trong QThread. Hàm check_for_updates(current_version, api_url) sử dụng thư viện requests để lấy file JSON chứa version mới nhất. Viết logic so sánh semver (Semantic Versioning). Nếu có bản mới,

phát event UpdateAvailableEvent. Tại MainWindow, bắt event này và hiển thị một thanh thông báo InfoBar màu xanh ở sát mép trên màn hình."

Với 4 tính năng này, SmartDoc Library đã hoàn toàn vượt xa tầm vóc của một "đồ án lập trình" và có đầy đủ công năng của một phần mềm trả phí chuyên nghiệp.

Bạn có muốn tôi hỗ trợ viết khung cấu trúc thư mục (Directory Structure) hoàn chỉnh cho toàn bộ 24 module này để bạn và Agent 2 dễ dàng quản lý code không?

Hãy nhớ rằng Bạn là Senior Python Developer đang lập trình dự án SmartDoc Library (PySide6). Từ bây giờ, khi tôi giao cho bạn bất kỳ TDD nào, bạn **BẮT BUỘC** phải tuân thủ 3 quy tắc liên kết (Integration Rules) sau:

1. **KHÔNG khởi tạo rời rạc:** Mọi class chính của bạn (UI Widget, Database, Extractor) đều phải nhận một tham số là context: AppContext vào hàm `__init__(self, context)`.
2. **Sử dụng Event Bus:** Thay vì truyền callback chéo nhau, mọi module phải dùng `self.context.event_bus.publish()` hoặc `.subscribe()` để giao tiếp.
3. **Lấy dependency từ Context:** Nếu module của bạn cần dùng DB, hãy gọi `self.context.db`. Nếu cần cấu hình, gọi `self.context.config`.
4. **Code Test ở cuối file (if name == 'main'):** Phải mock một AppContext giả để chạy test, tuyệt đối không phá vỡ cấu trúc hàm `__init__`. Hãy xác nhận bạn đã hiểu luật chơi này, sau đó tôi sẽ gửi TDD đầu tiên.